Dynamic-routing MVP findings — proxy-through-gluetun (Mullvad) e2e

Revision: 1 **Last modified:** 2026-07-02T18:30:00Z

Operator-authorized (2026-07-02): reset podman + prove the full proxy-through-gluetun Mullvad e2e. This document records the honest results (§11.4.6) — what is proven and the real MVP dynamic-routing defects that block the full squid-integrated path.

Proven live (rock-solid captured evidence)

- **podman/aardvark-dns is NOT broken** — throwaway inter-container DNS (`10.89.2.2`) and outbound DNS both resolve; the earlier "can't bind :53" fault has cleared. **No destructive `podman system reset` was done** — it would have destroyed the operator's 263 volumes (catalogizer/deploy/aosp/lava data — §9.2/§11.4.174).
- **squid-dynamic image rebuilt** (control-plane + squid multi-stage, `config/squid` context) → bakes `http_port :34128` (E's §11.4.108 staleness RESOLVED; source was already 34128).
- **Dynamic stack boots**; **gluetun WireGuard tunnel to Mullvad is UP** (exits observed Osaka / Denver / Malmö — public IP is a Mullvad exit each boot).
- **CORE proxy-through-gluetun egress PROVEN**: `http_proxy=http://proxy-gluetun:8888` → `am.i.mullvad.net/json` = `mullvad_exit_ip:true` (194.114.136.116, Osaka). Evidence `qa-results/dynamic_e2e_20260702T175536Z/`.
- **Squid dynamic-routing LOGIC proven**: un-routed target → fail-closed `TCP_DENIED/503 ERR_TUNNEL_DOWN` (§11.4.68); with a route:<host:port> + up-status the acl-helper returns `OK` tag=<tunnel> and squid flips to `TCP_TUNNEL` (allowed). The decision engine works.

Open MVP dynamic-routing DEFECTS (the full squid-integrated path does NOT yet pass)

#	Defect	Status
D1	Compiler renders `cache_peer gluetun-<profile>` but the MVP ships one `proxy-gluetun` → peers unresolvable → all routed targets 503.	**FIXED** `068dc78` (network aliases on the gluetun compose service).
D2	Route key is port-qualified: squid `%>ha{Host}` sends `host:443` on CONNECT, so routes must be `route:<host>:<port>`.	Documented (compiler/doc note).
D3	**healthd↔gluetun health signal BROKEN**: gluetun control API (`/v1/publicip/ip`, `/v1/vpn/status`) returns EMPTY; healthd's `wg show <if>` reads a WG interface in gluetun's netns (not healthd's) → every profile held falsely "down" → squid fail-closes even though the tunnel is genuinely UP.	**OPEN** (deepest — needs gluetun to expose the data OR healthd to read gluetun's wg stats).
D4	gluetun compose **healthcheck probes `/v1/openvpn/status`** for a WireGuard tunnel → gluetun labelled "unhealthy" though the WG tunnel is up.	**OPEN** (1-line compose fix to the WG status endpoint).
D5	Squid caches the `gluetun-*` peer **negative DNS** from its pre-alias startup; reconfigure did not clear it (peer stays dead). Needs squid to (re)resolve peers after gluetun is up (startup ordering / DNS lifecycle).	**OPEN**.

Honest conclusion (§11.4.6)

The proxy-through-gluetun **egress capability** is proven live (gluetun's proxy egresses via Mullvad). The **full squid-integrated** path is blocked by D3+D4+D5 (real integration bugs), so it is **NOT a passing e2e** — no PASS is claimed on it. D1 fixed; D2–D5 tracked here for a proper source-side fix pass.